Tutorial

Run the parts in this order, smallest first. Each step says what it proves and what you check with your own eyes. The first step that fails names the broken part.

Before you start

Nothing is generated. The run already exists, written by the earlier pipeline into `01-invoice-pipeline/Invoices/live/`:

File	Holds
`ledger.json`	every document the operation issued
`labels.csv`	the answer key: the error each invoice carries, the false positive it was built to draw, and how long the error took to come to light
`data/`	what the reader got out of the PDFs: 1,003 rows
`2026-01/` to `2026-10/`	the PDFs themselves
`erp/`	the vendors and their changes, the orders and their lines, the contracts, the receipts, the shipments, the rate card, the tariff table, the approval policy

The key holds 1,044 rows and 1,003 invoice numbers: 41 numbers were issued twice, revised or resent. One invoice is one number, so everything here counts numbers.

```
python -m pip install -r requirements.txt
```

Step 1: the whole suite

```
python -m pytest -q
```

Proves: every part works alone before anything works together.

Check by hand: everything passes. Nothing is skipped: a suite that quietly proves nothing is worse than one that fails. The retired tests under `tests/_retired/` are not collected; they measured the invented runs that have been dropped.

Step 2: build the table

```
python run.py prepare
```

Proves: the run loads as one set, the features are built from what was knowable on each invoice's own date, and the set is cut to a size 35 errors can carry.

Check by hand: it prints 1,003 rows and 35 errors. It prints the feature count before and after the cut, and names what it kept. The cut is about one feature per ten errors, so expect three.

Step 3: measure the models

```
python run.py train
```

Proves: the clerk's twelve checks are beaten, or they are not.

Check by hand:

- The split prints its cutoff and both sides. About two thirds of the errors train; the rest are tested on.
- Every figure has an interval beside it. A model whose interval overlaps another's has not been shown to differ from it.
- The clerk's checks are one row in the same table, measured the same way. **If they win, that is the result.**

Today they do win: average precision 0.53 against 0.13 for LightGBM, on ten errors, with every interval overlapping.

Step 4: write what the dashboard reads

```
python run.py score
```

Proves: every invoice gets a score, a flag, the check that explains it and the verdict on it.

Check by hand: 1,003 rows at `03-payables-panel/data/anomalies.csv`, each number once. Run it twice: the second run replaces the first, and the row count does not move.

Step 5: let it run

```
python run.py watch
```

Proves: the part a client is buying. Week by week: the model on duty ranks the week, the verdicts come back, they become labels, a fall or a drift starts a challenger, and a better one takes over.

Check by hand:

- Weeks come out in order, none twice.
- No week is scored by a model that was trained on it.
- Each retraining names what triggered it, precision or drift.
- **A challenger takes over only when the intervals separate.** With 35 errors that will rarely happen, and a promotion on overlapping intervals is the mistake this project exists to avoid.

Stop it whenever. `data/watch-state.json` keeps its place, and the next run carries on from the following week.

Step 6: the gauges

```
python run.py serve
```

Serves on port 9109 until stopped, and re-reads the scored file every 30 seconds. The monitoring stack of the earlier project scrapes it:

```
cd ../01-invoice-pipeline/monitoring
PIPELINE_HOST=$(ip route | awk '/default/ {print $3}') docker compose up -d
```

`PIPELINE_HOST` must be the Windows gateway. Without it Prometheus looks for port 9109 inside WSL, finds nothing, and the model panels stay empty while everything else works.

Step 7: the panels

The four panels are drawn on the cost dashboard, from `data/model-report.json`. Their shape, with invented numbers, is in panels-sketch.html:

Panel	Shows
confusion matrix	caught, missed, false alarm, left alone
precision against recall	every threshold, each model against the clerk
errors caught per invoice checked	how far down the ranking you go for each error found
errors caught by kind	which kinds are never found

When a number looks wrong

1. **The interval.** A figure that moved may not have moved. Check whether the intervals overlap before believing it.
2. **The split.** A model tested on weeks it trained on scores well and means nothing. The split prints its dates for this reason.
3. **The simulator.** Every invoice is invented. A result here is a result about the simulation, and the README says so.

Every command, file, feature, check and threshold: the reference. Where each number comes from: sources.